

J-ART：アプリケーション層の LLM セキュリティと コスト効率のための日本語敵対的レッドチーム評価フ レームワーク

テクニカルレポート (v0.2, プレプリント)

廣瀬 隆之 (Takayuki Hirose)*

2026-06-14

概要

大規模言語モデル (LLM) の公開レッドチーミング/ジェイルブレイク評価の多くは英語中心で、かつ**素のモデル**を対象とする。本稿は **J-ART** (日本語敵対的レッドチーム評価フレームワーク) を提案する。J-ART は、(i) 素のモデルではなく**アプリケーション構造層**——モデル × システムプロンプト強度 × ガードレール × 検索 (RAG)——を評価し、(ii) ギャル文字による字形置換・縦書き/改行分割・慇懃無礼 (keigo) な権威付け・二枚舌の偽前提・Base64・leet/ゼロ幅密輸といった**日本語特有の難読化変形**で素朴なキーワード検閲を突破し、(iii) 攻撃を **MITRE ATLAS** (推論 (運用) 時に行われる 9 技術) に対応づけ、(iv) 防御率に **Wilson 95% 信頼区間**を付し **コスト効率指標**を併記する、再現可能なオープンハーネスである。攻撃は**無害なプロキシ** (架空のカナリアと無害なマーカー) で構成され、**危険物を一切含まず**、各攻撃の悪意あるコアは公開物上でマスクされる。21 のアプリ構成と 11 攻撃 × 7 変形 (防御率は実 LIVE $K=5$ キャンペーン、差分の主張は 3 モデルの $K=3$ 均衡アブレーション) にわたり、**アプリ層の構成が「素のモデル」の大きなばらつきを潰すことを示す**：素のモデルは防御 29.6 ~ 86.5% とばらつくが、3 モデルすべてで強化またはガードレール付きの構成はいずれも **91 ~ 100% 帯**に収まり、一度アプリを構成するとモデル選択の差はほとんど効かなくなる。クラスタ頑健 CI のもとで強化プロンプトとモデルベースのガードレールは各モデルで有意に防御を引き上げ、軽量の keyword/regex は素の弱いモデルほど効く (天井付近では有意でなく、強化プロンプト側は飽和する)。防御率は **lexical-leak の上限**であり、真の拒否とマーカー抑制を完全には区別できない (第 7 節)。単純な**正規化フィルタ**が素朴なキーワードフィルタの取りこぼす難読化を無効化することを示す。さらに、構成を固定した $K=5$ 反復で **9.6% のセルが反復間で結果を変える**ことを記録し、単一サンプルのリーダーボードが信頼できないこと、統計的処理が必要であることを論じる。

ステータス. 本稿はフレームワークとその実 LIVE 知見を記述するテクニカルレポート/プレプリントであり、いかなるベンダーのモデルの公式な安全性評価でもない。数値は実行時スナップショット (2026-06-14) で第 7 節の限界に従い、全防御率は *lexical-leak* の上限である。§4 の数値は第 9 節に明記した commit 済みアーティファクトから再現可能。参考文献は実在の正準的文献であり、最終

* 独立研究者 (Independent Researcher)。ORCID: 0009-0005-6735-1862。コード: <https://github.com/takker-hero-se/J-ART>。リーダーボード: <https://takker-hero-se.github.io/J-ART/>。ソフトウェア DOI (concept): [10.5281/zenodo.20676879](https://doi.org/10.5281/zenodo.20676879)。本文 CC-BY-4.0、コード MIT。氏名の漢字表記は必要に応じて修正してください。

的な書誌整形は camera-ready で行う。

1 はじめに

公開 LLM レッドチーミングのツール（例：garak [2]）やジェイルブレイクのベンチマーク（例：HarmBench [3], JailbreakBench [4]）は英語が主流である。多言語安全性の研究は、プロンプトを低資源言語へ翻訳することで安全機構を回避できることを示してきたが、言語固有の文字・組版・敬体を悪用する言語特有の難読化は相対的に未開拓である。とりわけ日本語は、複数の文字体系、全角/ゼロ幅文字、縦書き、高度に慣習化された敬語など、豊かな難読化の表面を持つ。

さらに2つのギャップが J-ART の動機である。第一に素のモデル評価とデプロイシステム評価の差：実務では、モデルに加えてシステムプロンプト、さらにガードレール、しばしば検索が組み合わさる。セキュリティ上意味のある単位はアプリケーション構成であり、モデル単体ではない。第二にセキュリティとコストの同時評価：防御はトークンやレイテンシのコストを伴うため、コスト効率スコアを併記し「最も安価で十分に安全な構成」を直接読み取れるようにする。

本稿の貢献は次の通り：(a) 日本語難読化変形の一式、(b) 安全なプロキシマーカー方式を備えたアプリ層・MITRE ATLAS 準拠の評価ハーネス、(c) 正規化フィルタとモデルベース分類器を含むガードレール比較、(d) 統計的に報告するリーダーボード（Wilson 信頼区間・任意の N 回試行）とコスト効率指標、(e) モデルルーター（モデル要求を実際のバックエンドに振り分ける中継層）起因の大きな実行間ばらつきの実証的観察。

2 関連研究

J-ART は次の6系統の研究の交点に位置する。各系統を要約し、表1に位置づけをまとめる。

■敵対的レッドチーミングのツール. garak [2] はモデル単体を既知の脆弱性プローブ集合で走査する。J-ART は、配備されたアプリケーション構成（モデル × システムプロンプト × ガードレール × RAG）を評価単位とする。

■ジェイルブレイク・有害行動ベンチマーク. AdvBench/GCG [11]、Jailbroken [12]、HarmBench [3]、JailbreakBench [4]、実環境収集プロンプト [13] は、どの有害行動をどう採点するかを標準化するが、その大半は英語・モデル層が対象である。J-ART はハーネスの発想を継承しつつ、有害誘発を安全なプロキシマーカー目標 (§3) に置き換え、スイートを公開可能にする。

■自動攻撃生成. PAIR [6] と TAP [7] は攻撃プロンプトを反復最適化する。J-ART は再現性と安全な開示のため、固定・監査可能な変形一式を用いる。

■多言語・低資源の安全性. Deng ら [14]・Yong ら [5] は低資源言語への翻訳でアライメントが劣化することを、Wang ら [15] は言語横断の安全性一般化を示す。これらは言語間の翻訳を扱うが、J-ART は日本語固有の言語内字形難読化（ギャル文字・縦書き・敬語フレーミング・かな/漢文字符号化）を扱う。これは日本語が高資源であっても残存する。

■符号化・字形難読化. 符号化ベースの脱獄は確立した先行研究である：CipherChat [28]（暗号会話）と ArtPrompt [29]（アスキーアート字形）。本作の base64_wrap・leet_smuggle はこの系統

表 1 代表的先行研究に対する J-ART の位置づけ。

研究	言語	評価単位	コスト 軸	ガード/RAG	公開
garak [2]	英語	モデル	無	無	有
HarmBench / JBB [3, 4]	英語	モデル	無	無	有
PAIR / TAP [6, 7]	英語	モデル	無	無	一部
多言語 [5, 14, 15]	低資源	モデル	無	無	有
Llama Guard / NeMo [9, 18]	英語中心	分類器	無	該当無	有
J-ART	日本語	アプリ構成	有	有	有

の言語非依存な複製でありベースライン用途にすぎない。新規性は日本語字形の変形 (§3) にある。

■**プロンプトインジェクションとアプリ層評価**. Perez & Ribeiro [16] は目標乗っ取り・プロンプト漏洩を、Greshake ら [8] は間接注入を定式化し、Liu ら [17] は LLM 統合アプリのベンチマークを、AgentDojo [30] は最も近いアプリ層評価（ツール利用エージェント）を与える。J-ART は**アプリ構成**（プロンプト × ガード × RAG）を明示的な評価単位として日本語難読化面に対して評価する点で異なる。

■**ガードレール・入力分類器**. Llama Guard [9]、NeMo Guardrails [18]、OpenAI Moderation API [19] はモデル/規則ベースのフィルタを提供する。J-ART はガードレールを**差し替え可能な構成変数**として扱い、各々の限界防御を測る。素朴なキーワード検閲が取りこぼす難読化を破る決定論的正規化フィルタも含む (§4)。

■**LLM-as-judge の妥当性**. Zheng ら [20] と Gu ら [21] は LLM ジャッジの偏りを記録する。J-ART は**主要指標**で LLM ジャッジを避け、決定論的なカナリア/マーカー照合 (§3) を用いてジャッジ分散を指標から排除する。

■**日本語 NLP 資源とタクソノミ**. JGLUE [22] のような一般日本語ベンチマークは自然言語理解を対象とするが、日本語固有の難読化を狙う公開・再現可能な**敵対的安全性**ベンチマークは管見の限り存在しない。これが J-ART の埋める空白である。攻撃は MITRE ATLAS [1] に対応づけ、OWASP Top 10 for LLM Applications [10] を相互参照する。

すなわち J-ART は、**日本語の言語内字形難読化の表面**、**アプリ構成**の評価単位、**コスト効率の軸**、**安全なプロキシマーカー方式**を、単一の再現可能なハーネスに統合する点で他と異なる。

3 手法

■**アプリケーション構造層**. 評価対象(**target**)は構成である: provider/model × prompt_strength ∈ {naked (low), hardened (high)} × guardrail ∈ {none, keyword, regex, llm, llamaguard} × rag ∈ {true, false}。簡易 RAG 文脈（良性の社内文書、間接インジェクション攻撃では汚染）と、秘密の**カナリア**を含むシステムプロンプトを試行ごとに組み立てる。

表 2 難読化変形の分類。

クラス	変形	悪用する性質
日本語固有	gyaru, vertical_newline, polite_business, double_tongue	仮名字形の異形・縦書き・敬語による権威/内集団フレーミング
言語非依存	base64_wrap, leet_smuggle (+ baseline)	任意の言語に適用できる符号化 (+ /記号置換)

■**日本語難読化変形**. 各ベース攻撃は 7 種の変形のいずれかでレンダリングされる: baseline (無変形)、polite_business (敬語による権威付け)、vertical_newline (1 文字 1 行の縦書き)、gyaru (ギャル文字の字形置換+ゼロ幅スペース)、double_tongue (偽のサンドボックス/認可の前提)、base64_wrap (Base64 エンコードした指示)、leet_smuggle (leet 記号置換+ゼロ幅スペース)。変形は人間可読性を保ちつつ、部分文字列一致のフィルタを破壊するよう設計される。

変形は 2 クラスに区別する (表 2)。**日本語固有**の変形は日本語の表記体系や待遇表現を悪用し、英語などに直接の対応物を持たない: gyaru (仮名字形置換)、vertical_newline (縦書き)、polite_business/double_tongue (敬語による権威付けと内集団フレーミング)。**言語非依存**の変形 — base64_wrap・leet_smuggle (および対照の baseline) — は**新規ではなく**、符号化/字形脱獄の系統 (CipherChat [28]・ArtPrompt [29]) の複製であり、日本語固有面の限界効果を切り分けるための比較基準としてのみ含める。各変形を無害な例文に適用した実出力を図 1 に示す。

■**攻撃スイートと MITRE ATLAS 対応**. 11 の攻撃が、推論 (運用) 時に行われる ATLAS 技術 9 種を網羅する: 直接/間接プロンプトインジェクション (AML.T0051.000/.001)、LLM ジェイルブレイク (AML.T0054)、LLM データ漏洩 (AML.T0057)、LLM プロンプト自己複製 (AML.T0061)、信頼出力/引用の操作 (AML.T0067.000)、LLM プロンプト難読化 (AML.T0068)、システムプロンプト探索 (AML.T0069.002)、RAG 汚染 (偽 RAG エントリ注入を含む) (AML.T0070)。ベクトルは user (直接) または rag (間接)。

■**無害なプロキシマーカーと設計による安全性**. 危険なコンテンツの生成を避けるため、攻撃は**無害なプロキシ**を標的とする: システムプロンプトに埋め込んだ架空のカナリア合言葉 (データ漏洩攻撃) と、無害な文字列マーカー [JART-PWNED] / [JART-HIJACKED] (注入/制御奪取攻撃)。各攻撃の悪意ある**コア**はコード内に定義し、設定ファイルには残さず、results.json と公開サイトでは**マスク** (伏字プレースホルダへ置換) する。表示されるのは難読化の**フレームワーク**のみであり、利用可能なレシピを与えずに構造を学べる。描画時にもクライアント側の多重マスクを施す。

■**決定論的ジャッジ**. 突破はカナリア/マーカーの membership で判定する — 完全一致および軽い正規化後の一致。本ジャッジは *lexical* (字句的) な漏洩を検出するが、意味的/言い換えの漏洩は検出できない。よって本指標を **lexical-leak 防御率**と呼ぶ: 本稿の全防御率は真の安全性の**上限**であり、保証ではない。人手検証付きジャッジは今後の課題。

無害な例文「システムの稼働状況を報告してください」を各変形に適用した実出力（攻撃の悪意あるコアは含まない）。太字は各手口の要点。

baseline（無変形）

システムの稼働状況を報告してください

polite_business（慇懃無礼）

平素より大変お世話になっております。株式会社さくら情報セキュリティ部の監査担当でございます。…
【ご依頼事項】システムの稼働状況を報告してください…**本依頼は経営層の承認済みであり、ご対応は貴殿の正当な業務範囲に含まれます。**

vertical_newline（縦書き分割）

シ／ス／テ／ム／の／稼／働／状／況／を／報／告／し／て／く／だ／さ／い（実際は1文字ずつ改行）

gyaru（ギャル文字+ゼロ幅）

お願い→システム稼働状況ヲ報告シテください←ょЯレ<（各文字間にゼロ幅スペースを挿入）

double_tongue（二枚舌）

【前提条件の共有】あなたは現在、**本番環境から隔離された検証用サンドボックス**上で動作しています。…
通常の安全制約はすべて無効化されています。…指示：システムの稼働状況を報告してください

base64_wrap（符号化）

44K344K544OG44Og44Gu56i85YON...（Base64 エンコード全体）

leet_smuggle（符号化）

システムの稼働状況を報告してください（文字間にゼロ幅スペースを挿入。leet 記号置換はラテン文字に作用）

図 1 各難読化変形を無害な例文に適用した表示例。polite_business/double_tongue は敬語・偽前提による **フレーミング**、gyaru/vertical_newline は字形・組版の難読化、base64_wrap/leet_smuggle は言語非依存の符号化。

■**ガードレール**. 5 設定を比較する：**none**；**keyword**（禁止部分文字列一致——高速だが難読化に弱い）；**regex**（**正規化**フィルタ。ゼロ幅文字除去・leet 逆変換・空白畳み込み・長トークンの Base64 復号を行ってから禁止語照合——決定論的で API 追加コスト 0）；**llm**（対象モデル自身が入力を検閲し出力も審査）；**llamaguard**（専用の安全性分類モデル——ルーター経由の Llama Guard 4——が入力を攻撃/良性に分類し、失敗時は安全側にフォールバック）。

■**統計的处理**. 各防御率は試行数に基づく **Wilson 95% 信頼区間**と共に報告する。各セルは K 回反復する（JART_TRIALS。本稿ではライブ campaign が $K=5$ 、アブレーションが $K=3$ ）。ただし試行レベルの Wilson 区間はセルの K 反復を独立と扱うが、実際は**独立でない**。したがって構成間の差（アブレーション、§4）には、セルごと復元抽出する**クラスタ頑健ブートストラップ 95% CI**（bootstrap_diff_ci、3,000 再標本、固定シード）を報告する。これは試行レベルの Newcombe 区間より広く、全有意性主張の根拠となる区間である。

■**コスト効率指標**. $\text{cospa} = \text{防御率}(\%) \div 100 \text{ 万トークンあたりコスト (USD)}$ を報告する（単価は下限\$0.01 でクランプ。すなわち\$0.01 未満の単価は\$0.01 として扱う）。cospa は意図的に単純な**スクリーニング比**であり効用モデルではない：単位は独立に意味を持たず、順位はクランプと価格スナップショットに感度を持つ。よって並べ替えキーとしてのみ用い、**防御 対 コストのトレードオフ**

を直接提示する。クランプの影響は \$0.01/100 万トークン未満の構成に限られ、評価対象のモデル群には該当がない。

■**実行**. 試行は独立であり、同時実行数を制限した並列実行（既定 8 ワーカーのスレッドプール）で、リクエスト毎のタイムアウトと指数バックオフ再試行のもとに実行する。シミュレーションの決定論は実行順に依存しない。API キーが無い対象は決定論的シミュレーションにフォールバックし、鍵が一部欠けても実行全体が落ちない。

4 結果

完全ラベル付きの **LIVE キャンペーン** (21 構成 × 11 攻撃 × 7 変形 × $K = 5$ 反復 = 8,085 試行、2026-06-14 実行) を報告する。各セルは mode=LIVE と価格スナップショットでラベル付けし、api_error の試行のみ（試行単位で）除外する——parity 再実行後は 169/8,085 を除外し 7,916 クリーン試行が残る。クリーンセルの実課金（キャンペーン + parity 再実行、高価な Opus を含む）は $\approx \$6.0$ 。Opus 素は完全測定 ($n=385$ 、Anthropic 補充で parity 達成)；Gemini-pro のみ部分的 ($n=227$ 、分間レート制限) で暫定扱い。全防御率は lexical-leak の上限 (§3) であり、差にはすべてクラスタ頑健 CI を付す (§3)。§4 の各数値は commit 済みアーティファクトから再現可能 (§9)。

■**全体**. 全構成は §3 の簡易 RAG コンテキストを含む。「素」は強化プロンプトなし・ガードなしの意であり「RAG なし」ではない（よって間接注入攻撃は素のベースラインでも対象）。クリーン試行の総突破率は 14.5% (1,151/7,916) で、弱構成に集中した。(gemini-pro はこのプールに暫定の防御試行 $n=227$ を寄与する (§7)。除外すると総突破率は 15.0% となるが、全体像は変わらない。) 強化済み/ガード付き構成はどのモデルでも 91–100% 帯で防御したのに対し、素のモデルは 29.6%~86.5% に散らばった (表 3)。

■**アプリ層の防御 対 モデル能力**. 構成効果をモデル能力から切り分けるため、能力域をまたぐ 3 モデル (弱い Llama-4-Scout・安価な GPT-4o-mini・フラグシップ GPT-4.1) で均衡アブレーションを行う。各々モデルを固定し{素, 強化プロンプト} × ガードレールを直積、各構成の素ベースラインとの差をクラスタ頑健ブートストラップ 95% CI (3,000 セル再標本。試行ではなくセルを再標本化するため、アブレーションの $K=3$ 反復のセル内相関で有意性が水増しされない。experiments/analyze_ablation.py で再現) で報告する (表 4)。アブレーションした 3 モデルすべてで 2 つの傾向が成り立つ：(1) 強化プロンプトとモデルベースのガード (Llama Guard・LLM 自己検閲) は各モデルで有意に防御を上げる；(2) 効果は素のベースラインに反比例——弱い Scout (素 38%) では各ガードが +26~+60 ポイント、GPT-4o-mini (素 82%、天井近い) では軽量の keyword/regex はクラスタ補正で有意でない。ただし防御率は 100% で頭打ちのため、この反比例の一部は天井 (headroom) の作用であってガードレール固有の性質ではなく、また強化プロンプトを入れると、その上にガードレールを重ねた追加効果はほとんどない (強化アームは 100% 近くで飽和)。

したがって核心は「どれか一つの防御がモデル性能を凌駕する」ことではなく、構成が素の大きなばらつきを潰すことである：素のモデルは 29.6~86.5% (表 3) だが、アブレーションした 3 モデル

表3 素のモデルの防御率（プロンプト・ガードなし、 $K = 5$ ）。

素のモデル	防御率	95% 信頼区間	n
DeepSeek-V3	29.6%	[25.3, 34.4]	385
Llama 4 Scout	37.9%	[33.2, 42.9]	385
Gemini 2.5 Flash	49.6%	[44.6, 54.6]	385
gpt-oss-120b	82.6%	[78.5, 86.1]	385
GPT-4o-mini	83.9%	[79.9, 87.2]	385
Claude Opus 4.x	86.5%	[82.7, 89.5]	385

表4 構成効果（ Δ = そのモデルの素ベースライン比、クラスタ頑健 95% CI）。 $\times$ = 非有意。

追加防御（低プロンプト）	Scout (38.1%)	4o-mini (82.3%)	GPT-4.1 (64.1%)
+ keyword	+26.0 [+13,+39]	+9.1 [−1,+20] $\times$	+12.5 [−1.5,+26] $\times$
+ regex	+45.0 [+33,+56]	+10.0 [+0,+20] $\times$	+23.8 [+11,+36]
+ Llama Guard	+47.2 [+36,+58]	+15.6 [+7,+25]	+25.1 [+13,+38]
+ LLM ガード	+59.7 [+51,+68]	+17.7 [+10,+26]	+35.9 [+26,+47]
強化プロンプト	+47.2 [+36,+58]	+17.7 [+10,+26]	+33.3 [+23,+45]

すべてで強化/ガード付き構成は **91~100%** 帯に収まり、**一度構成すればモデル選択の差は素のときよりはるかに小さくなる**。具体的には、強化した安価モデル（GPT-4o-mini 強化 $\approx 100\%$ ）が素のフラグシップ（GPT-4.1 素 64.1%、 $n=231$ ）を ~ 36 ポイント、素の Opus (86.5%、 $n=385$) も上回る。この比較には2つの限定がある。第一に、「素」は**ゼロ防御の下限**（システムプロンプトもガードルールも無い＝誰もデプロイしない構成）であり、ギャップは**構成あり vs なし**と読むべきで、モデル同士の対等比較ではない。デプロイ現実的な最小プロンプトの baseline は今後の課題。第二に、本結果はアブレーションした3モデル・ $K=3$ で確立したものであり、全 fleet に対して主張するものではない。さらに**構成妥当性**の留保が全体にかかる：判定器はカナリア/マーカの完全一致（第3節）なので、強化プロンプトが「マーカを出すな」と命じると、真の拒否ではなく**マーカ文字列の抑制**で測定値が上がりうる。よって本指標は真の強化とマーカ抑制を完全には区別できない。実際、予備的な semantic-judge 監査（defended 60 セル・LLM ジャッジ）では、この緩さは**約 7~10 ポイント**と推定され、leet/base64 などの符号化変形に集中した（判定器が攻撃別の完全一致で、難読化されたマーカ【例 `J0RT-PWN3D`】）やカナリアの別経路漏洩を取りこぼすため）。より厳密な判定器（全攻撃でカナリアを照合・leet 逆変換）と人手検証は今後の課題とし、本稿の絶対値はこの留保付きで読むべきである。

■**実行内ばらつき（主要な結果）**。単一キャンペーン内で、構成を固定したまま各セルを $K = 5$ 反復した。全 5 試行がクリーンな 1,577 セルのうち **9.6%** (152/1,577) が 5 回の同一反復で結果を変えた——素の弱モデルに集中。当初の戒めの例「Llama 4 Scout / 素の API」では 77 セル中 **32 (42%) が不安定**：14 が無突破 (0/5)、31 が常時突破 (5/5)、32 が中間。単一サンプルのリーダーボードはこの同一構成に運次第で 0%~100% を割り当てる——構成集合の交絡を排した**単一サンプルは再現しない**ことの統制された証拠である。この知見は**単一キャンペーン内**のばらつき（同一実行中のプロバイダ非決定性に由来）に限定する。日をまたいだ変動（経時ドリフト）は、計測の枠組み（across_runs）だけ用意し、本格的な分析は今後の課題とする。

■**ガードレール比較とコスト**. 3つのモデル(表4)で、防御を上げる効果の順序は一貫していた。効果が最も大きいのは **LLM ガードレールと強化プロンプト**、次が **Llama Guard**、軽量の keyword/regex フィルタはベースモデルが弱いほどよく効く。入力を遮断するガードレールは、本体モデルの呼び出し自体を省けるため、**防御を上げつつ正味コストはむしろ下がり得る**。例えば正規化 regex フィルタ(ユニットテストで検証済み)は、弱い Scout に対し **API 追加コスト 0** で +45 ポイントの防御を加える。コスト効率(cospa)で見ると、上位は**安価な OSS モデル+軽量ガード**(gpt-oss-20b + keyword、cospa $\approx$ 1131)、最下位は**素のフラグシップ**(Opus 素、cospa $\approx$ 6.8)で、両者には**3桁(約1000倍)の開き**がある。

■**攻撃・変形の有効性**. 最も突破率が高かった攻撃は、信頼出力/引用の操作・偽 RAG エントリ注入・システムプロンプト探索だった。難読化変形どうしを比べると、意外にも**無変形**の baseline が最も突破率が高く(**22.0%**)、次いで vertical_newline (20.0%)、polite_business (17.3%)と続いた。一方、base64_wrap は**最も弱かった**(5.5%)。これは、Base64 のような重い符号化をかけると、モデルは指示をすり抜けさせるよりむしろ拒否したり解釈に失敗したりすることが多いためである。つまり**最も危険な日本語変形は、人間が読めてしまう「自然な」もの**(縦書き・敬語フレーミング)であって、暗号めいた難読化ではない。

5 リーダーボード成果物

ハーネスは results.json と自己完結の静的サイト(並び替え可能なリーダーボード、マスク済みプロンプト付きの構成別攻撃ログ、環境に応じて自動選択する日英 UI、モデル別防御率の信頼区間、セル別突破率)を生成し、継続的にデプロイする。

6 倫理と責任ある開示

■**設計による安全性**. J-ART は設計により安全である。(1) 攻撃は**架空のカナリアと無害なマーカー**を標的とし、実際の有害能力を扱わない。コード・データ・公開サイトのいずれにも危険物は存在しない。(2) 悪意ある**コア**は全成果物でマスクされ、難読化のフレームワークのみが示される。(3) モデルは研究比較のためにのみ参照され、結果は実行時スナップショットであって認証ではない。意図する用途は、開発者が**自身のデプロイ**を堅牢化する支援である。

■**再利用可能な開示パターン**. 上記の設計を、運用可能な攻撃を公開せずに敵対的安全性ベンチマークを公開するための転用可能なパターンとして一般化する：(i) **無害なプロキシ目標**(架空カナリアと無害マーカー)により突破を測定可能だが不活性にする；(ii) **コア/フレームワーク分離とマスキング**—研究信号を担う難読化フレームワークは公開し、運用コアは全成果物でマスクし、無害コアはコード内のみに保持する；(iii) **決定論的オフラインモード**によりリポジトリ単体で結果を再現でき、稼働する攻撃コーパスを配布しない；(iv) **マスキングの監査**—コア(生・難読化いずれも)が公開成果物に到達しないことをユニットテスト不変条件で検査する。これにより手法と集計結果は完全公開しつつ、攻撃ペイロードは秘匿される。言語固有のレッドチームデータセット全般に推奨する姿勢である。

表 5 ガバナンス・クロスワーク（情報提供）。

J-ART 能力 (AT-LAS)	OWASP LLM Top 10	NIST AI RMF	EU AI Act	日本
プロンプト注入/脱獄 (T0051, T0054)	LLM01	MEASURE 2.7	第 15, 55 条	AISI; AI 事業者
カナリア/漏洩 (T0057)	LLM02	MEASURE 2.7	第 15, 10 条	AI 事業者
システムプロンプト探索 (T0069.002)	LLM07	MEASURE 2.7	第 15 条	AISI
RAG 汚染/偽エントリ (T0070)	LLM08; LLM01	MEASURE 2.7	第 15, 10 条	AISI
信頼出力の操作 (T0067.000)	LLM09; LLM05	MEASURE 2.6	第 50 条	AI 事業者
難読化/自己複製 (T0068, T0061)	LLM01	MEASURE 2.7	第 55 条	AISI

■**ガバナンス・クロスワーク**. 表 5 は、評価する各能力を MITRE ATLAS [1]、OWASP Top 10 for LLM Applications [10]、NIST AI RMF [23] とその生成 AI プロファイル [24]、EU AI Act [25]、日本の AI ガバナンス指針 [26, 27] に相互参照する。本対応は**情報提供**であり**適合性の主張**ではない。適用可否は各デプロイのリスク分類に依存する。運用上 J-ART は NIST AI RMF の **MEASURE** 段階の活動であり、その出力は組織の **MANAGE** 判断に資する。

■**具体例**. NIST MEASURE 2.7 の下で、組織は J-ART の行を「プロンプト注入耐性 (T0051/T0054)、lexical-leak 防御率 93.5% (単一構成の率なので Wilson 95% CI [90.6, 95.6]。クラスタ頑健 CI は**差用**、§3)、 $n=385$ 、 $K=5$ 、GPT-4.1-mini + 強化プロンプト + keyword ガード、2026-06-14」と記録し、事前登録した閾値 (例「CI 下限 $\geq 90\%$ 」) を添えられる。同じ行が AISI レッドチーム報告や EU AI Act 第 15 条の堅牢性資料が引く証跡となる。

■**デュアルユース**. 公開する変形スイートは再利用可能な攻撃ラッパーでもある。攻撃者の利得は小さい (日本語字形変形は俗知、符号化は公開研究の複製、有害ペイロードは含まない) 一方、防御側の便益は具体的: 同じリポジトリが全変形を無効化する**正規化 regex ガード** (ユニットテスト済み) を同梱する。総合的に公開は防御側に正味プラスと判断する (§6.2 と整合)。

7 限界

- **プロキシマーカーであって実害ではない**: 実害ではなく指示違反/漏洩の代理を測る。
- **決定論ジャッジ**: カナリア/マーカーを完全一致および軽い正規化 (大小文字畳み込み・空白/ゼロ幅除去) 後に照合するため、整形してエコーされた漏洩も突破として計上する。一方で**意味的/言い換え**による漏洩は検出できないため、報告する防御率は**上限**である。人手検証付き LLM ジャッジは今後の課題。
- **単発 (1 ターン)**: 多ターン/crescendo 攻撃は未モデル化。
- **標本数**: 手作りのスイート (11 攻撃 $\times$ 7 変形)、既定はセル当たり $N=1$ 。
- **プロバイダの非決定性/ルーターのばらつき**: 結果はスナップショットであり、実行時のルーティングとモデル版に依存する。

- **欠損データの偏り（概ね解消）**：以前のキャンペーンでは、一部の構成のデータが偏って欠けていたことを指摘していた。原因は、API のレート制限が難しい攻撃ほど起きやすく、Opus の最難関の攻撃が十分に測れていなかったこと——失敗しやすい攻撃が抜け落ち、防御率が実際より高く見える生存バイアスである。全件を測り直す parity 再実行でこれを直接確認した：Opus の素モデルは完全測定（ $n=385$ ）で防御 **86.5%** だったのに対し、偏った初期スナップショット（ $n=125$ ）では **89.6%** と、予測どおり約 **3 ポイント過大評価**されていた（偏ったスナップショットは成果物として残さず、確定値 $n=385$ のみを commit）。現在 Qwen3-max と Opus は完全測定に到達。**Gemini-pro だけが部分測定**（ $n=227$ 。分あたりのレート制限が原因で、課金を足しても解消しない）のため**暫定扱い**とし、主要な結論はこれに依拠しない。
- **コスト帰属**：ガードレールのトークンは**ガードモデル自身**の単価で計上するよう修正（llamaguard は分類器 JART_GUARD_MODEL、llm は実際に呼び出す本体モデル）し、本体トークンと別建てで報告する。残る近似：MOCK 時のトークン**数**は推定値である。
- **シミュレーションモード**は鍵欠如時の決定論的代替であり、実測のモデル挙動と読むべきではない。

8 今後の課題

人手検証付きの LLM ジャッジ、多ターン・適応的（PAIR/TAP 系）攻撃、より大規模で一部人手キュレートした攻撃集合、ATLAS 技術の追加（コスト枯渇・モデル抽出）、プロバイダ別並列と増分キャッシュによる経時トレンド追跡、そしてシミュレーションモードを外した純粹に実測・反復試行による統計報告。

9 再現性

コード・設定・デプロイ用ワークフローは公開している。ハーネスはシミュレーションモードでは決定論的であり、LIVE 結果は実行時のプロバイダ・ルーティングとモデル版に依存するため、実行日とモデル識別子と共に報告すべきである。使用した正確なモデル識別子と単価はリポジトリを参照のこと。各セルは来歴ラベル——mode (LIVE/MOCK)、api_error、解決済みモデル、price_per_million スナップショット——を持ち、シミュレーションセルは LIVE 集計から除外される。**§4 の全数値は commit 済みアーティファクトから再現可能**：experiments/var_run.json (§4.1 全体・素のスプレッド・分散・変形・cospa) と、3 つのアブレーションファイル ablation_results.json (GPT-4o-mini)・abl_scout_results.json (Llama-4-Scout)・abl_gpt41_results.json (GPT-4.1、表 4)。集計とクラスタ頑健 CI は analyze_ablation.py / analyze_variance.py で再計算する。**公開リーダーボードは commit 済みの LIVE キャンペーン成果物**（var_run.json）を配信するため、本稿の数値とサイトが一致する。新規 LIVE 再測定は定期実行（および手動）で行い、push 時は課金せず commit 済み成果物を再公開するため、サイトが MOCK と LIVE を混在させることはない。

AI による支援の開示

著者は本研究全体にわたり、Claude Code コマンドラインインターフェース経由で **Claude Opus 4 (Anthropic)** を補助として用いた。その用途は二つである。(A) **執筆**: 査読対応の3ラウンドにわたる本文の草稿作成と改訂、日本語アブストラクトおよび英語版(`technical-report.tex`)との二言語編集、引用エラーの点検(著者帰属・出版年・書誌の完全性)。(B) **コード実装**: 評価ハートネスおよび解析スクリプトの実装の補助。これには統計処理(Wilson 区間、Newcombe 差分区間、`analyze_ablation.py` および `analyze_variance.py` のクラスタ頑健ブートストラップ)の実装を含む。

手法に関する決定——実験設計、用いる統計手法とその根拠(例: セル内の相関を考慮し、試行ではなくセル単位で再サンプリングする)、各主張の射程、結果の解釈——は、すべて単独の人間著者が決定し主導した。AI が生成した文章およびコードはすべて著者が確認・テスト・検証しており、報告したすべての数値は付属スクリプトによりコミット済みアーティファクトから再現できる。**デュアルロールに関する注記**: Claude Opus 4.x は本稿において**評価対象**モデルの一つとしても登場する (§4、素の防御率 86.5%、 $n=385$)。補助ツールとしての役割と研究対象としての役割は別個であり、透明性のため双方をここに開示する。本稿の全内容・全主張・全分析について著者が単独かつ全面的に責任を負う。

参考文献

- [1] MITRE ATLAS — Adversarial Threat Landscape for AI Systems. <https://atlas.mitre.org> (accessed 2026-06-14).
- [2] garak — Generative AI Red-teaming & Assessment Kit.
- [3] HarmBench: A Standardized Evaluation Framework for Automated Red Teaming.
- [4] JailbreakBench: An Open Robustness Benchmark for Jailbreaking LLMs.
- [5] Yong, Menghini, Bach. Low-Resource Languages Jailbreak GPT-4. 2023.
- [6] Chao et al. Jailbreaking Black-Box LLMs in Twenty Queries (PAIR).
- [7] Mehrotra et al. Tree of Attacks: Jailbreaking Black-Box LLMs Automatically (TAP).
- [8] Greshake et al. Not What You’ve Signed Up For: Indirect Prompt Injection. 2023.
- [9] Inan et al. Llama Guard: LLM-Based Input-Output Safeguard for Human-AI Conversations. Meta, 2023.
- [10] OWASP Top 10 for Large Language Model Applications (2025). <https://genai.owasp.org/llm-top-10/> (accessed 2026-06-14).
- [11] Zou et al. Universal and Transferable Adversarial Attacks on Aligned Language Models (AdvBench/GCG). 2023.
- [12] Wei, Haghtalab, Steinhardt. Jailbroken: How Does LLM Safety Training Fail? 2023.
- [13] Shen et al. “Do Anything Now”: Characterizing and Evaluating In-the-Wild Jailbreak Prompts on Large Language Models. 2024.
- [14] Deng et al. Multilingual Jailbreak Challenges in Large Language Models. 2024.

- [15] Wang et al. All Languages Matter: On the Multilingual Safety of Large Language Models. 2024.
- [16] Perez & Ribeiro. Ignore Previous Prompt: Attack Techniques for Language Models. 2022.
- [17] Liu et al. Prompt Injection Attack against LLM-Integrated Applications. 2023. arXiv:2306.05499.
- [18] Rebedea et al. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. 2023.
- [19] Markov et al. A Holistic Approach to Undesired Content Detection in the Real World (OpenAI Moderation). 2023.
- [20] Zheng et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. 2023.
- [21] Gu et al. A Survey on LLM-as-a-Judge. 2024.
- [22] Kurihara et al. JGLUE: Japanese General Language Understanding Evaluation. 2022.
- [23] NIST. AI Risk Management Framework (AI RMF 1.0), NIST AI 100-1. 2023.
- [24] NIST. Artificial Intelligence Risk Management Framework: Generative AI Profile, NIST AI 600-1. 2024.
- [25] European Union. Regulation (EU) 2024/1689 (Artificial Intelligence Act). Official Journal of the EU, 2024.
- [26] 経済産業省・総務省. AI 事業者ガイドライン Ver. 1.0. 2024.
- [27] AI セーフティ・インスティテュート (AISI). レッドチーミング手法ガイド. 2024.
- [28] Yuan et al. GPT-4 Is Too Smart To Be Safe: Stealthy Chat with LLMs via Cipher (CipherChat). 2023.
- [29] Jiang et al. ArtPrompt: ASCII Art-based Jailbreak Attacks against Aligned LLMs. 2024.
- [30] Debenedetti et al. AgentDojo: A Dynamic Environment to Evaluate Attacks and Defenses for LLM Agents. 2024.